



深圳大学
Shenzhen University

互联网编程

第9章

客户端Socket

Socket帮互联网应用开发的程序员做了什么事情？

Internet数据传输

- Internet数据按有限大小的包传输，数据报(datagram)
- 数据报=首部(header)+有效载荷(payload)
- 首部：包括发送方的地址和端口、接收方的地址和端口、检测数据是否被破坏的检验和、保证可靠传输的其它管理信息。
- 有效载荷：数据本身。
- 需分解包、错误检测、数据重组、重传包、重排序等工作



Socket

- Socket封装了两个主机之间的一个连接
- 支持7个操作：
 - 连接远程主机
 - 发送数据
 - 接收数据
 - 关闭连接
 - 绑定端口
 - 监听入站数据
 - 在绑定端口上接受来自远程机器的连接
- Java的Socket类提供了前4个操作方法，客户端和服务端都可以使用
- 后3个操作，是等待客户端的连接，仅服务器需要，由ServerSocket类实现。



Java的Socket实现

■ TCP协议

- 提供端到端面向连接的可靠的全双工字节流传输服务

■ **Java.net.Socket**类

- 连接远程主机、关闭连接
- 收、发数据
- 常见用法
 - 创建Socket
 - Socket连接远程主机

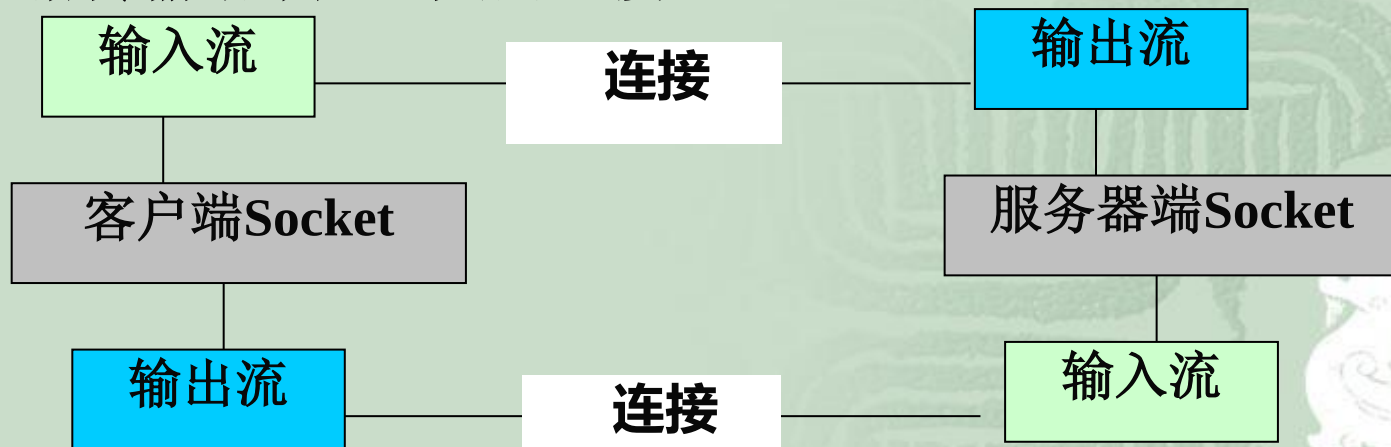
■ **Java.net.ServerSocket**类

- 绑定、监听端口
- 处理端口上的连接请求
- 收、发数据



Java程序如何使用客户端Socket

- java程序用Socket构造函数创建一个新的Socket连接实例
- Socket尝试连接远程主机，连接成功就可从socket得到**输入流**和**输出流**
- Java程序使用输入流和输出流相互发送数据
- 连接是全双工的：客户端和服务端可以同时发送和接收数据
- 传输的数据的含义取决于应用层协议：
- 先完成协商握手，再具体传输数据
- 数据传输结束后，关闭连接



Socket连接示意图

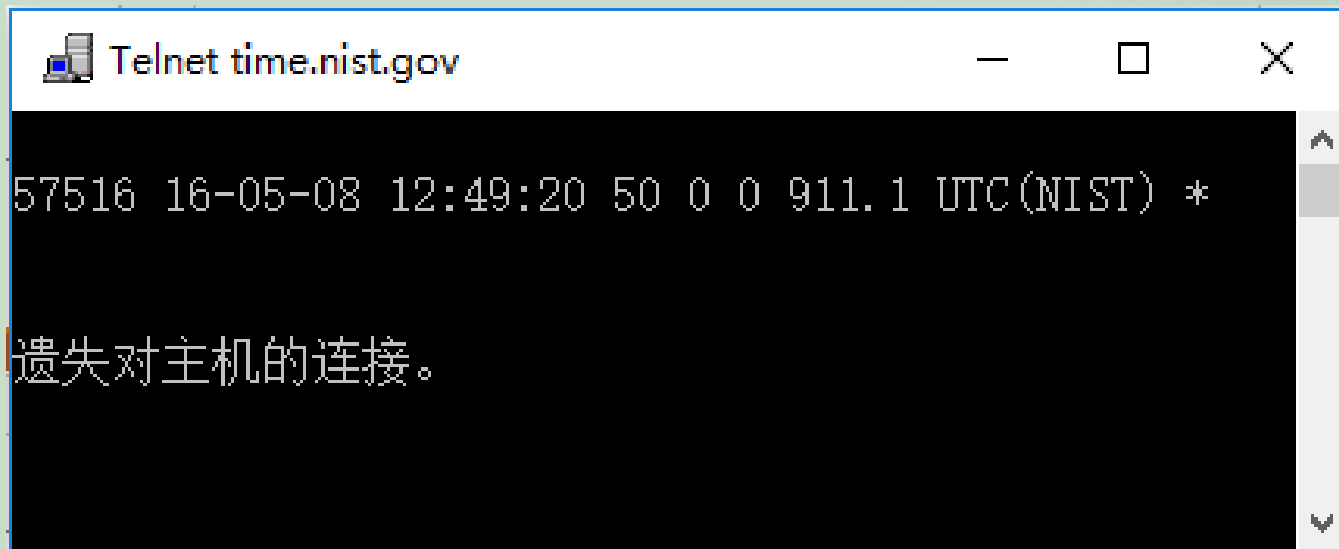
Telnet协议

- 工作在应用层的一个Internet协议
- Telnet
 - WinXP及之前默认开启；Win7及之后默认关闭；
 - 如何开启Telnet
 - 控制面板 -> 程序和功能 -> 打开或关闭Windows功能 -> 勾选Telnet客户端
- 可以使用Telnet命令连接服务器
 - 如：telnet www.szu.edu.cn 80
- 是否能够连通，取决于服务器是否支持telnet协议



用telnet命令方式从时间服务器读取数据

- 连接美国国家标准技术研究院（NIST）的时间服务器，请求当前时间：
 - telnet time.nist.gov 13
 - 服务器会以RFC867协议、人可读的格式返回结果；非标准时间服务协议



```
Telnet time.nist.gov

57516 16-05-08 12:49:20 50 0 0 911.1 UTC(NIST) *

遗失对主机的连接。
```



日期格式

Daytime服务器发送的时间:

57516 16-05-08 12:48:05 50 0 0 105.0 UTC(NIST) *

- 读取此连接此服务器的socket的InputStream，可得到此结果。时间格式为：

JJJJJ YY-MM-DD HH:MM:SS TT L H msADV UTC(NIST) OTM

- JJJJJ: 修正儒略日（Modified Julian Date）自1858.11.17以来的日数
- YY-MM-DD HH:MM:SS 日期与时间
- TT: 指示采用标准时间还是美国夏令时，50美国夏令时；00标准时间
- L: 一位码，指示当前月最后一天子夜是滞增加或减去一个闰秒。0表示无闰秒，1表示增加三个闰秒，2表示减去一个闰秒
- H: 服务器状态，0表示健康，1指最多相差的5秒，2指相差超过5S
- msADV: 延迟补偿毫秒数，NIST把此数增加到它发送的时间,对网络延迟做补偿
- UTC: Coordinated Universal Time

使用socket编程方式从时间服务器获取数据

- Java7以上
- ```
try (Socket socket= new Socket("time.nist.gov",13)){
 // 从socket读取.....
}
catch (IOException ex){
 System.err.println("could not connect to time.nist.gov");
}
```
- ```
Socket socket=null;  
try{  
    socket=new Socket("time.nist.gov",13);  
    // 从socket读取...  
}catch (IOException ex){  
    System.err.println(ex);  
}finally{  
    if (socket!=null){  
        try {  
            socket.close();  
        } catch (IOException e) { //doNothing }  
    }  
}
```

Java6及以前的版本
需要



为socket连接设置超时时间

- 对socket设置一个超时时间，如果连接服务器间服务器挂起，则会给出socketTimeoutException异常通知
 - ▣ `Socket.setSoTimeout(15000);//15秒`
`InputStream in = socket.getInputStream();`
 - ▣ //从输入流读取服务器发送的字节，此处的协议指定发送的字节必须是ASCII
`InputStreamReader reader=new InputStreamReader(in, "ASCII");`
`for(int c = reader.read(); c!=-1; c=reader.read()){`
 - ▣ `Time.append((char c);`
 - ▣ `}`
 - ▣ `System.out.println(time);`

演示编程例子ex8-1,ch8.DaytimeClient.java，应用客户端Socket实现Daytime协议客户端



```
3 import java.net.*;
4 import java.io.*;
5 public class DaytimeClient {
6     public static void main(String[] args) {
7         String hostname = args.length > 0 ? args[0] : "time.nist.gov";
8         Socket socket = null;
9         try {
10             socket = new Socket(hostname, 13);
11             socket.setSoTimeout(15000);
12             InputStream in = socket.getInputStream();
13             StringBuilder time = new StringBuilder();
14             InputStreamReader reader = new InputStreamReader(in, "ASCII");
15             for (int c = reader.read(); c != -1; c = reader.read()) {
16                 time.append((char) c);
17             }
18             System.out.println(time);
19         } catch (IOException ex) {
20             System.err.println(ex);
21         } finally {
22             if (socket != null) {
23                 try {
24                     socket.close();
25                 } catch (IOException ex) {
26                     // ignore
27                 }
28             }
29     }
```

DaytimeClient.java

30 }

31 }

使用协议和理解数据格式编程

- 网络程序的重点工作通常是使用协议和理解数据格式
- 如果不只是显示输出时间服务器发送给你的文本，而是希望把它解析为一个`java.util.Date`对象，该如何编程？

演示编程例子ex8-2， `ch8.Daytime.java`，与时间服务器`time.nist.gov`对话，将时间服务器返回的时间字符串转换成易读的时间格式`Date`对象



通过与时间服务器time.nist.gov对话构造一个Date

Daytime.java

```
3 import java.net.*;
4 import java.text.*;
5 import java.util.Date;
6 import java.io.*;
7 public class Daytime {
8     public Date getDateFromNetwork() throws IOException, ParseException {
9         try (Socket socket = new Socket("time.nist.gov", 13)) {
10             socket.setSoTimeout(15000);
11             InputStream in = socket.getInputStream();
12             StringBuilder time = new StringBuilder();
13             InputStreamReader reader = new InputStreamReader(in, "ASCII");
14             for (int c = reader.read(); c != -1; c = reader.read()) {
15                 time.append((char) c);
16             }
17             return parseDate(time.toString());
18         }
19     }
20
21     //将字符串转换为日期
22     static Date parseDate(String s) throws ParseException {
23         String[] pieces = s.split(" ");
24         String dateTime = pieces[1] + " " + pieces[2] + " UTC";
25         DateFormat format = new SimpleDateFormat("yy-MM-dd hh:mm:ss z");
26         return format.parse(dateTime);
27     }
28 }
```


编程处理服务器返回的复杂数据格式

- 网络程序遇到不同的协议，可能会遇到陌生复杂的数据格式，那么程序必须读取原始字节，再按字节解释。
- 必须自己编写代码处理服务器发送的任意格式的数据

演示编程例子ex8-3, ch8.Time.java, 时间协议客户端



```
3 import java.net.*;
4 import java.text.*;
5 import java.util.Date;
6 import java.io.*;
7 public class Time {
8     private static final String HOSTNAME = "time.nist.gov";
9     public static void main(String[] args) throws IOException, ParseException {
10         Date d = Time.getDateFromNetwork();
11         System.out.println("It is " + d);
12     }
13
14     public static Date getDateFromNetwork() throws IOException, ParseException {
15         // The time protocol sets the epoch at 1900,
16         // the Java Date class at 1970. This number
17         // converts between them.
18         long differenceBetweenEpochs = 2208988800L;
19         // If you'd rather not use the magic number, uncomment
20         // the following section which calculates it directly.
21         /*
22         TimeZone gmt = TimeZone.getTimeZone("GMT");
23         Calendar epoch1900 = Calendar.getInstance(gmt);
24         epoch1900.set(1900, 01, 01, 00, 00, 00);
25         long epoch1900ms = epoch1900.getTime().getTime();
26         Calendar epoch1970 = Calendar.getInstance(gmt);
27         epoch1970.set(1970, 01, 01, 00, 00, 00);
28         long epoch1970ms = epoch1970.getTime().getTime();
```

```
29
30 long differenceInMS = epoch1970ms - epoch1900ms;
31 long differenceBetweenEpochs = differenceInMS/1000;
32 */
33 Socket socket = null;
34 try {
35     socket = new Socket(HOSTNAME, 37);
36     socket.setSoTimeout(15000);
37     InputStream raw = socket.getInputStream();
38
39     long secondsSince1900 = 0;
40     for (int i = 0; i < 4; i++) {
41         secondsSince1900 = (secondsSince1900 << 8) | raw.read();
42     }
43     long secondsSince1970
44         = secondsSince1900 - differenceBetweenEpochs;
45     long msSince1970 = secondsSince1970 * 1000;
46     Date time = new Date(msSince1970);
47     return time;
48 } finally {
49     try {
50         if (socket != null) socket.close();
51     }
52     catch (IOException ex) {}
53 }
54 }
55 }
```

用socket写入服务器

- 用Socket写入服务器，需向socket请求一个输出流和一个输入流,使用输出流在socket上发送数据，同时使用输入流读取数据。
- 使用RFC2229的字典(dict)协议：
 - ▣ telnet dict.org 2628
 - ▣ 服务器正常，字典有变更，可以使用show db查看db，然后使用类似
 - ▣ define fd-eng-fra XXXX 来查询单词
 - ▣ 编程实现该协议客户端：

```
Socket socket = new Socket("dict.org",2628);
OutputStream out = socket.getOutputStream();
Writer writer = new OutputStreamWriter(out, "UTF-8");
writer.write("DEFINEENG-latgold\r\n");
writer.flush();
```

演示例子代码ex8-4,ch8.DictClient.java一个基于网络的英语-拉丁语翻译程序，dict协议应用的客户端实现



```
3 import java.io.*;
4 import java.net.*;
5 public class DictClient {
6     public static final String SERVER = "dict.org";
7     public static final int PORT = 2628;
8     public static final int TIMEOUT = 15000;
9     public static void main(String[] args) {
10         Socket socket = null;
11         try {
12             socket = new Socket(SERVER, PORT);
13             socket.setSoTimeout(TIMEOUT);
14             OutputStream out = socket.getOutputStream();
15             Writer writer = new OutputStreamWriter(out, "UTF-8");
16             writer = new BufferedWriter(writer);
17             InputStream in = socket.getInputStream();
18             BufferedReader reader = new BufferedReader(
19                 new InputStreamReader(in, "UTF-8"));
20
21             for (String word : args) {
22                 define(word, writer, reader);
23             }
24             writer.write("quit\r\n");
25             writer.flush();
26         } catch (IOException ex) {
27             System.err.println(ex);
28         } finally { // dispose
29             if (socket != null) {
```




```
30     try {
31         socket.close();
32     } catch (IOException ex) {
33         // ignore
34     }
35 }
36 }
37 }
38
39 static void define(String word, Writer writer, BufferedReader reader)
40     throws IOException, UnsupportedEncodingException {
41     writer.write("DEFINE eng-lat " + word + "\r\n");
42     writer.flush();
43
44     for (String line = reader.readLine(); line != null; line = reader.readLine()) {
45         if (line.startsWith("250 ")) { // OK
46             return;
47         } else if (line.startsWith("552 ")) { // no match
48             System.out.println("No definition found for " + word);
49             return;
50         }
51         else if (line.matches("\\d\\d\\d .*")) continue;
52         else if (line.trim().equals(".")) continue;
53         else System.out.println(line);
54     }
55 }
56 }
```


半关闭socket

- **Close()**方法同时关闭**socket**的输入和输出。
- 若只需要关闭连接的一半如输入或者输出，则可用：
 - 可以关闭一半的Socket
 - shutdownInput ()/ shutdownOutput()
 - 后续仍然需要close
 - 半关闭方法实质并未关闭**socket**，只是调整与**socket**连接的流，使之认为已经到了流的末尾。半关闭方法并不释放与**socket**关联的资源，如端口等。关闭输入之后再读取输入流会返回-1，关闭输出之后再写入**socket**会抛出一个**IOException**异常
 - 即使半关闭了连接，或将连接的两半都关闭，使用结束后仍需关闭该**socket**
 - 若需确定输入流和输出流是打开的还是关闭的，可使用：

```
Public boolean isInputShutdown()  
Public boolean isOutputShutdown()
```



构造和连接Socket

- `Java.net.socket`类是java完成客户端TCP操作的基础类。使用原生代码与主机操作系统的本地TCP栈进行通信。
- 其他建立TCP连接的面向客户端的类，如`URL`、`URLConnection`、`Applet`和`JEditorPane`，最终都会调用这个类的方法：
 - 基本构造函数
 - 选择从哪个本地接口连接
 - 构造但不连接
 - `Socket`地址
 - 使用代理服务器
 - 获取`Socket`的信息
 - 关闭还是连接



基本构造函数

- `public Socket(String host, int port)` throws `UnkownHostException`, `IOException`
- `public socket(InetAddress host, int port)` throws `IOException`
- 第一个构造方法多了`UnkownHostException`的异常
- 利用构造函数创建`Socket`对象，连接远程主机`Socket`
 - 端口扫描（攻防第一步，通常被防火墙阻拦、报告）
 - ```
for(int i=1;i<1024;i++){
 try{
 Socket s = new Socket(host, i);
 //doSomething
 s.close();
 }.....
```

演示编程例子 **ex8-5**, `ch8.LowPortScanner.java`, 检测指定主机上哪些端口开放，安装有**TCP**服务



程序实现：查看指定主机上前1024个端口中哪些安装有TCP服务器

LowPortScanner.java

```
1 package ch8;
2
3 import java.net.*;
4 import java.io.*;
5
6 public class LowPortScanner {
7
8 public static void main(String[] args) {
9 System.out.println("Start scanning...");
10 String host = args.length > 0 ? args[0] : "localhost";
11 System.out.println("host is " + host);
12 for (int i = 1; i < 1024; i++) {
13 try {
14 Socket s = new Socket(host, i);
15 System.out.println("There is a server on port " + i + " of "
16 + host);
17 s.close();
18 } catch (UnknownHostException ex) {
19 System.err.println(ex);
20 break;
21 } catch (IOException ex) {
22 // must not be a server on this port
23 }
24 }
25 }
26 }
```

# 选择从哪个本地接口连接

- `public Socket(String host, int port, InetAddress interface, int localPort)` throws `IOException`, `UnknownHostException`
- `public Socket(InetAddress host, int port, InetAddress interface, int localPort)` throws `IOException`
- 前2个参数：远程主机；
- 后2个参数：本地接口和端口选择
- 本地接口接口和端口异常会抛出如`BindException`等 `IOException`的子类





# 构造但不连接

- ❑ `public Socket()`
- ❑ `public void connect(SocketAddress endpoint, int timeout) throws IOException`
- 先构造，以后再连接的编程方式：

```
■ Socket socket=new Socket();
 SocketAddress address= new InetSocketAddress(SERVER, PORT);
 try{
 socket.connect(address);
 // doSomething
 }catch (IOException ex){ //doSomething
 finally{
 try {
 socket.close();
 } catch (IOException ex) { ... }
 }
```



# SocketAddress

- 当前只支持 TCP/IP Socket，都是InetSocketAddress的实例
  - ▣ public SocketAddress getRemoteSocketAddress()
  - ▣ public SocketAddress getLocalSocketAddress()
- 用于socket.connect
- 也可使用InetSocketAddress直接构造
  - ▣ public InetSocketAddress(InetAddress address, int port)
  - ▣ public InetSocketAddress(String host, int port)
  - ▣ public InetSocketAddress(int port)
  - ▣ public static InetSocketAddress createUnresolved(String host, int port)
- InetSocketAddress提供了getAddress、getPort、getHostName等方法



# 代理服务器

- `public Socket(Proxy proxy)`
- 默认由`socksProxyHost`和`socksProxyPort`系统属性控制
- 可以通过前述函数自定义
  - `Proxy.NO_PROXY`，不使用代理
  - `Proxy proxy = new Proxy(Proxy.Type.SOCKS, proxyAddress)`
  - `Proxy.Type.HTTP`
  - `Proxy.Type.DIRECT`



# 获取Socket的信息

- Socket对象有一些属性:

- 远程地址
- 远程端口
- 本地地址
- 本地端口

- 可以通过以下获取方法来访问:

- `public InetAddress getInetAddress()`
- `public int getPort()`

- `public InetAddress getLocalAddress()`
- `public int getLocalPort()`



# 关闭还是连接

- 判断一个socket是否连接到一个远程主机
  - `public boolean isConnected()`
    - ▣ Returns the connection state of the socket.
    - ▣ 返回true, socket确实能够连接远程主机, 即使socket已关闭。
  - 判断socket的状态: 是否关闭
  - `public boolean isClosed()`
    - ▣ Returns the closed state of the socket.
    - ▣ 返回true, 已关闭; false:未关闭, 没连接过也会是false
- 检查一个socket是否打开的正确方式:
  - `boolean connected = socket.isConnected() && !socket.isClosed();`



# 设置Socket选项

- Socket选项指定了java socket类所依赖的原生 socket如何发送和接收数据。客户端socket, java支持9个选项
  - TCP\_NODELAY
  - SO\_LINGER
  - SO\_TIMEOUT
  - SO\_RCVBUF和SO\_SNDBUF
  - SO\_KEEPALIVE
  - OOBINLINE
  - SO\_REUSEADDR
  - IP\_TOS服务类型





# TCP\_NODELAY

- 设置TCP\_NODELAY为true可确保包会尽可能快地发送
  - `public void setTcpNoDelay(boolean on) throws SocketException`
  - `public Boolean getTcpNoDelay() throws SocketException`
- 为了提高传输效率，Nagle算法进行了发送缓冲
- 在某些特定情况下不希望缓冲
  - `Socket.setTcpNoDelay(true);`



# SO\_LINGER

- SO\_LINGER选项指定了Socket关闭时如何处理尚未发送的数据报，
- 默认情况下close()方法将立即返回，但系统仍会尝试发送剩余的数据。
  - `public void setSoLinger(boolean on, int seconds) throws SocketException`
  - `public int getSoLinger() throws SocketException`
- 0：不等，丢弃
- 正数：阻塞close，等n秒
- getSoLinger等于-1表示还没设置过



# SO\_TIMEOUT

- 设置SO\_TIMEOUT可以确保此次调用阻塞的时间不会超过某个固定的毫秒数。
  - ▣ `public void setSoTimeout(int milliseconds) throws SocketException`
  - ▣ `public int getSoTimeout() throws SocketException`
- 只需要read，通常都要设置超时时间
- 一次read超时，并不会关闭连接
- 0意味着没有超时时间（连接正常时会一直等）



# SO\_RCVBUF和SO\_SNDBUF

- 接收和发送缓冲区的大小
- 给底层实现的建议
- 主要还是由TCP滑动窗口机制进行控制
  - 太小不能有效利用网络
  - 太大会阻塞网络



# SO\_KEEPALIVE

- `public void setKeepAlive(boolean on) throws SocketException`
- `public boolean getKeepAlive() throws SocketException`

- 你还活着吗？
- 一般2小时一次，向空闲连接发送探测数据包，无响应则持续尝试11分钟（通常75秒一次），都没响应则关闭连接



# OOBINLINE

- 设置是否希望接收正常数据中的紧急数据
- 默认情况下，java会忽略从socket接收的紧急数据
  - `public void setOOBInline(boolean on) throws SocketException`
  - `public boolean getOOBInline() throws SocketException`
  - `public void sendUrgentData(int data) throws IOException`
- 收发紧急数据
- 实际支持并不是很好
- 用于在正常输入流中，插入有特异性的“紧急数据”，而且依赖高层应用程序的特别处理，例如“**ctrl+c**”





# SO\_REUSEADDR

- 绑定端口会独占端口
- 端口关闭时网络上仍然可能会传输发给旧端口的数据
- 因此通常要等一小段时间端口才恢复可用
- SO\_REUSEADDR，不等
  - 先构造socket但不连接（无参数构造函数）
  - 再设置setReuseAddress(true)
  - 再connect
  - 之前的Socket也需要这样设置才能生效



# IP\_TOS服务类型

- 设置IP报头的服务类型字段



# SOCKET异常

## ■ IOException

### □ SocketException

#### ● BindException

- ✓ 通常是端口被占用，也可能是无权限使用该端口

#### ● ConnectException

- ✓ 连接被远程主机拒绝，通常是远程主机没在监听对应端口或进程忙

#### ● NoRouteToHostException

- ✓ 连接超时

### □ ProtocolException

- 数据不符合 TCP/IP 规范

